TSseek: Regular Expression-Based Similarity Search for Distributed Time Series Datasets

Xiaoshuai Li[+], Khalid Alnuaim[+], Mohamed Y. Eltabakh[*], Elke A. Rundensteiner[+]
[+]Worcester Polytechnic Institute, Worcester, MA, USA
{xli3, kalnuaim, rundenst}@wpi.edu
[*]Qatar Computing Research Institute, Doha, Qatar
meltabakh@hbku.edu.qa

ABSTRACT

Similarity search is a fundamental operation in time series analysis. Most existing techniques, however, require users to supply a precise sequence of values (typically an entire time series object) as the query input. This rigid requirement limits real-world applications, where users instead want to express patterns, trends, or value ranges. Flexible, pattern-based search has been explored in text retrieval and complex event processing, but remains underexplored for large-scale distributed time series. To close this gap, we propose **TSseek**, a regular-expression-powered search framework for distributed time series datasets. TSseek's query language enables users to compose patterns encompassing trends, value ranges, and wildcard segments. We show that conventional approximation techniques (e.g., PAA and SAX) and their index structures are ill-suited for such queries because they cannot operate on regular-expression query constructs. In TSseek, we map the time series objects and the query constructs into the same space by approximating time series objects as sequences of line segments that retain both trend (slope direction) and value range, and translating query constructs into bounding rectangles. To support efficient processing, we build **TSseek-X**, a distributed spatial index over the time series segments. TSseek supports two fundamental query types, namely whole-matching queries (over entire series) and subsequence-matching queries (over arbitrary windows within a series). Across benchmark and real-world datasets, full-scan, model-based, and SAX-based baselines all sacrifice either accuracy or speed, whereas TSseek returns exact answers efficiently. Also, for subsequence workloads, it achieves significant speedups over state-of-the-art subsequence matching engines.

KEYWORDS

Time series indexing, similarity search, pattern-based search, distributed processing

1 INTRODUCTION

Time series data are pervasive across a diverse array of domains, including finance, healthcare, scientific research, the Internet of Things (IoT), and sensor networks [10, 12, 17, 27, 43, 58, 59, 69]. The complexity and variety of tasks in these fields highlight the vital role of time series analysis methods, such as classification, clustering, prediction, anomaly detection, and forecasting [29, 47, 50, 57, 77]. At the heart of such time series analysis lies the fundamental operation of *similarity search*, which facilitates the identification of similar time series objects within a data set.

Similarity search approaches fall into two primary categories: *whole matching*, which compares complete time series data [4, 9, 34, 60], and *sub-sequence matching*, which searches for specific segments within a series [14, 38, 72, 82]. TSseek covers both categories. What sets TSseek apart is *what* the query can express, i.e., a unified regex-inspired vocabulary of singletons, value ranges, trend patterns (with optional length and magnitude constraints), and wildcards, composable into expressive queries. Each comparable time series similarity system covers only a narrow slice of this design space—TARDIS [79] and SEAnet [68] accept only a full-sequence value query and support only whole-matching; whereas T-ReX [28] accepts pattern-based subsequence queries but executes via full-pass scans without a persistent index.

Traditional approaches in both categories commonly use query-by-example, where users provide a precise sequence of readings to search for. In the context of whole matching, most existing approaches require the entire time series as input query [20, 46, 60, 79–81]. While pattern-based search has been explored in other domains (e.g., shape-based queries in centralized systems and CEP event patterns [18, 45, 78]), it remains an overlooked area for expressive pattern-based search over large-scale distributed time series datasets. This limitation restricts flexibility for many real-world applications, especially when users seek to express patterns, trends, or value ranges rather than exact value sequence loop-ups.

The need for a more expressive query language arises not only from practical application demands but also from the intrinsic properties of time series data. These data are typically high-dimensional, often consisting of hundreds or thousands of readings in each object, and exhibit value proximity that renders exact matches unnecessary in practice. For example, temperature readings of 90 and 89 may effectively be equivalent for many applications. To accommodate this approximation, existing similarity-search techniques employ k-nearest neighbors (kNN) and related distance-based queries [7, 19, 41, 79–81]. In TSseek, we give the flexibility to the end-user during query construction instead of pushing it to the processing algorithm.

Motivating Example: *Searching for and analyzing specific patterns in financial time series data can improve the understanding of financial market dynamics, help in investment risk management, and inform sophisticated investment strategies. Identifying such patterns may lead us to discover actionable insights that could guide decision-making and the development of financial products.*

Consider a financial analyst, Stephen, tasked with identifying stocks that exhibit a specific pattern of price movements: a consistent price increase over a period, followed by a stabilization period, and then a decline. Recognizing this pattern in large-scale historical stock data enables him to pinpoint entry and exit opportunities, helping investors maximize returns and minimize risks. By identifying less obvious patterns, Stephen seeks to exploit market inefficiencies and reduce uncertainty. To do this effectively, he requires a flexible yet

robust query tool empowered by a pattern-matching language that allows Stephen to express fine-grained constraints such as specifying a 5% increase over four months, followed by two months of stability, and a 3% drop thereafter. The ability to adjust these parameters allows exploration of pattern variations in large-scale datasets.

To address these limitations, we propose a framework for supporting regular-expression-based search over distributed time series datasets. This requires us to address two main challenges: **(1) Compatible Feature Extraction for Query and Dataset Objects.** TSseek's query objects are pattern-based representations, not standard time series objects, expressed at a different abstraction level than the dataset they target—they cannot be directly compared without an intermediate representation. We design a feature extraction mechanism that produces compatible features from both kinds of object—explaining why existing methods such as SAX [37] and PAA [31] are ineffective in our context. Because the same feature representation must serve as the substrate for the indexing and query-processing layer (Challenge 2 below), it must also be amenable to spatial indexing. **(2) Distributed Scalable Query Processing.** Time series data, often massive, is often stored across an array of distributed data servers. Further, pattern-based search introduces significant computational complexity, and straightforward methods like finite state automata are computationally prohibitive at scale. Therefore, TSseek necessitates the development of efficient indexing structures and scalable query processing algorithms that can handle millions of time series efficiently.

In this paper, we introduce **TSseek**, a scalable framework for regular-expression-based similarity search on distributed time series datasets. TSseek's query constructs let users compose patterns from trends, value ranges, singletons, and wildcards into either whole-matching queries (spanning the entire stored series) or subsequence-matching queries (matching a contiguous window inside an unconstrained host series) [4, 9, 34, 60].

During preprocessing, TSseek approximates each time series as a sequence of line segments enriched with slope and value-range metadata and indexes them spatially. At query time, each construct is mapped into a bounding rectangle over the same segment space; a two-tier prune-and-refine pipeline retrieves candidate series via spatial intersection and then validates each candidate with a finite-state-automaton (FSA) refinement step. The same pipeline supports subsequence-matching with two specializations: a segment-level composite filter (instead of per-construct probes) and a windowed-enumeration DFA (instead of position-locked validation).

In a nutshell, our work makes the following contributions:

• **Pattern-powered query constructs for time series similarity search.** We identify the limitation of existing similarity search techniques for time series data, which often require an actual time series object (i.e., a precise sequence of values) to be presented as the query object. To address this, we introduce query constructs adopted from regular expressions that allow users to define a broader range of pattern types within the query object.

• **Compatible feature extraction and indexing mechanism.** We propose segmentation-based feature extraction that generates compatible and comparable representation for both the time series data and the pattern-based query object. Such compatibility makes it feasible to construct **TSseek-X**, our distributed spatial index over these segments, and leverage it for efficient search.

• **Efficient Query Processing & Search Algorithms** We employ a distributed index-based prune-and-refine search algorithm over our segment representation that is applicable for both whole-match and subsequence-match queries, with an additional segment-level pruning specific to the subsequence case.

• **Extensive Experimental Study** We conduct extensive experiments on several benchmark and real world datasets and demonstrate that existing whole-match methods either fail to support pattern-based queries or have unacceptable results' accuracy below 30%. Moreover, TSseek achieved more than 10× speedup compared to the state-of-the-art subsequence match engines.

Remainder of Paper: In Sec. 2, we present an overview of related work and its limitations. Sec. 3 introduces the preliminaries and problem statement. In Sec. 4 and 5, we present TSseek's query constructs and index structure, respectively. The query processing technique is presented in Sec. 6. Finally, the experimental evaluation and concluding remarks are included in Sec. 7 and 8, respectively.

2 RELATED WORK

Whole and Sub-Sequence Similarity Search: Similarity search is a core operation in many time series analysis tasks [61]. Existing methods fall into two primary categories: *whole matching*, e.g., [4, 9, 34, 60], and subsequence matching, e.g.,[14, 38, 72, 82]. In both categories, prevailing search strategies require users to specify an exact sequence of time series values—either a complete time series object or a segment thereof [20, 46, 71]. As discussed in Sec. 1, this approach is often inflexible, limiting users' ability to express more abstract or general patterns within query objects. TSseek addresses this limitation by introducing an expressive yet user-friendly search paradigm.

Only a few studies have explored pattern-based search in time series data [23, 28, 39, 56, 75]. However, unlike TSseek, these methods are often tailored to specific use cases. For example, the method in [23] is designed for detecting *head-and-shoulder* patterns in financial datasets, while SAXRegEx [75] focuses on symbolic pattern matching in multivariate automotive sensor data. It focuses on handling certain distortion types that commonly exist in automotive time series datasets. RelaQ [39] also addresses pattern-based time-series querying, but through a visual GUI for analyst-driven exploration of inter-series relations (correlation, causality, lag) rather than scalable pattern retrieval.

Pattern-Based Subsequence Matching. The closest existing system to our subsequence pipeline is **T-ReX** [28], a segment-based subsequence pattern search engine that allows users to express pattern queries through segment variables and compositional operators. T-ReX evaluates queries directly over raw series with a tree-based executor and a cost-based optimizer; it does not maintain a persistent disk-based index, so every query incurs full-pass cost over the entire collection. TSseek's subsequence extension instead reuses the same offline segment index built for whole-matching, and we adopt T-ReX as the primary subsequence baseline in our evaluation.

Exact vs. Approximate Similarity Search: Independent of the whole vs. sub-sequence classification, similarity search techniques

can also be distinguished as either *exact* or *approximate*. In exact search, the result set is guaranteed to satisfy the query object and associated criteria (e.g., exact match or k-nearest neighbors) [19, 34]. In contrast, approximate search may yield results that include false positives or false negatives relative to the exact result set—typically in exchange for improved indexing efficiency and faster query processing [41, 65, 67]. Neither approximate nor kNN methods fulfill TSseek's objectives, as none expose user-defined pattern or trend/value-range constructs; TSseek is thus complementary to them.

Regex over Other Data Types: Pattern and regular expression (regex) search has been studied for other data types, e.g., text retrieval [45] and complex event processing (CEP) [18, 78]. Such techniques are not directly applicable to time series, however, due to fundamental differences in structure and semantics. For example, regex in text retrieval is optimized for character sequences and lacks support for numerical trends such as *"a sequence of ten increasing values"*. Similarly, CEP techniques are tailored for streaming and real-time data, making them unsuitable for the disk-based time series datasets assumed in TSseek and previous methods [26, 40].

Feature Extraction & Indexing Techniques: Many feature extraction techniques have been proposed in the literature including DFT [20], DWT [13], PAA [31], SAX [37], and iSAX [60]. Associated with each representation, index structures have been designed ranging from traditional Spatial Access Methods (SAMs) such as the Rtree [25] or its variants [5, 8, 30] to custom-designed indexes such as iSAX Binary Tree [11] and *sigTree* [79]. Notably, these feature extraction and indexing approaches are designed with the assumption that the query object is an exact sequence of time series values. As a result, they are not compatible with the TSseek query paradigm, which relies on pattern-based query objects that cannot be directly transformed into representations such as PAA or SAX. Consequently, TSseek introduces its novel feature extraction and indexing framework tailored to operate effectively within this new query setting.

Deep Learning-Based Similarity Search Models: Another emerging direction, departing from symbolic feature-based indexing, leverages deep representation learning to model semantic similarity in time series data [21, 68]. For example, SEAnet [68], a contrastively trained autoencoder is designed to preserve semantic proximity in the embedding space, while Series2Vec [21], a self-supervised framework, is designed to capture both time and frequency domain characteristics. These new approaches, as we have demonstrated experimentally, face several challenges that preclude them from deployment in practice, including sensitivity to the training data quality and domain, not scaling well to large real-world-sized time series datasets, and not natively supporting the pattern-based queries proposed in TSseek.

Full-Fledged Time Series Database Systems: Popular time series database engines, such as Apache IoTDB [66], InfluxDB [6], and TrajStore [70], provide highly efficient storage and compression mechanisms, high-throughput data ingestion and processing, and support for online aggregations and analytical tasks. However, these systems do not support the regular-expression search capabilities TSseek provides.

3 TSSEEK PRELIMINARIES

Definition 1. Time Series: A time series $X =< x_1, x_2, \ldots, x_m >$, where $x_i \in \mathbb{R}$ for all $1 \leq i \leq m$, is an ordered sequence of m real-valued variables. We assume that the readings arrive at fixed time granularities, and hence timestamps are implicit.

Definition 2. Time Series Dataset: A time series dataset $\mathcal{T} = \{X_1, X_2, \ldots, X_n\}$ is a collection of N time series objects X_i, each of length m, meaning, all objects have the same length.

The proposed query constructs in TSseek enable users to express their queries using regular expressions by embedding patterns, value ranges, direction, and wild card segments into the query objects. A TSseek query object consists of one or more of these constructs as follows.

Definition 3. TSseek Query Object: A TSseek query object Q consists of a sequence of TSseek query constructs $Q =< q_1, q_2, ..., q_k >$, where each construct q_i has an associated *type* and *location* (start and end positions). The length of each construct q_i is determined by $|q_i|$ = (q_i.end - q_i.start + 1). The length of the overall query object $|Q| = \sum_{i=1}^{k} |q_i| \leq m$, with m the length of TS objects in the dataset; whole-matching queries are the special case $|Q| = m$, and subsequence-matching queries are the general case $|Q| < m$, where Q must match some contiguous window of the host series.

The types and properties of the query constructs will be presented in detail in Sec. 4.2. Per Def. 3, our work supports both *whole-matching* queries ($|Q| = m$) and *subsequence-matching* queries ($|Q| < m$); whole-matching is the more common formulation in the literature [3, 20, 36]. Unlike previous work where the precise m values must be defined, in TSseek, only the non-wildcard constructs must be explicitly defined, i.e., gaps (missing segments) in the query object are implicitly assumed to be wildcard constructs.

Example 1: *An example TSseek query object starts with ten precise values x_1, ..., x_{10}, followed by an increasing twenty values falling in the range between 50 and 100, then a sequence of decreasing five values, and the rest can be anything. This query object consists of a sequence of four constructs; one for the fixed values of length 10, followed by an increasing trend of length 20, and then a decreasing trend of length 5, and finally the rest is a do-not-care (wildcard) segment.*

Definition 4. Problem Statement: Given a TSseek query object Q and a time series dataset $\mathcal{T}$, find all TS objects $X \in \mathcal{T}$ that matches Q (whose syntax is formally defined via query constructs in Sec. 6).

Per Def. 4, we support *exact similarity search*, similar to previous work in the literature [3, 32, 34, 60]. However, the advantage of TSseek over existing work is that its query object Q is pattern-based, and hence it is more expressive compared to a strict sequence of precise values adopted in previous work.

4 SYSTEM OVERVIEW

4.1 TSseek System Architecture

In Fig. 1, we illustrate the high-level system architecture of TSseek, which operates in two distinct modes: **offline data processing** and **online query processing**.

In the offline mode, datasets undergo feature extraction, index construction, and distributed storage. While our focus is on static

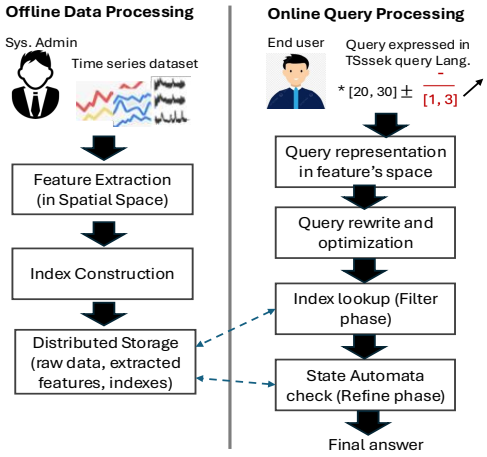


Figure 1: TSseek System Architecture.

datasets, the framework can be naturally extended to support incremental batch updates. In the online mode, users formulate queries using TSseek's pattern language. These queries are transformed into representations aligned with the extracted feature space, after which they pass through a query rewriting and optimization stage prior to execution. As depicted in Fig. 1, query execution proceeds in two phases: (i) **a filtering phase**, which employs the constructed indexes to retrieve a set of candidates ensuring that all matches are safely contained within the intermediate result set, and (ii) **a refinement phase**, which leverages a State Automaton for the final correct answer set.

Common Pipeline for whole and subsequence matching. Both modes go through the identical two-phase architecture above (filter then refine) over the same offline segment index. They differ only in the form of the filter-phase lookup (a per-construct probe set for whole-matching versus one composite scan for subsequence-matching) and in the placement check the refine-phase DFA performs (a position-locked test versus a windowed enumeration under ordering and gap constraints). Sec. 6 gives the algorithmic detail for both modes.

4.2 TSseek Query Constructs and Language

A promising strategy for designing an expressive pattern-based query language for time series retrieval is to leverage regular-expression-inspired constructs. Regular expressions, widely recognized for their expressiveness, have been extensively applied in various domains, including text search [16, 63], complex event processing (CEP) [18, 45, 78], and programming languages [2, 22, 52]. Over the years, regular expressions have evolved to include a rich set of advanced expressive constructs [42, 48, 52].

In TSseek, we adopt a minimalist approach by incorporating a limited set of constructs capable of capturing common time series patterns. This design choice enables an end-to-end exploration of the system's key components, including efficient feature extraction, indexing, and query processing techniques. Future work will focus on enriching the query language with more sophisticated pattern constructs.

TSseek currently supports four primary types of query constructs that can be parameterized to allow flexible and expressive pattern specification. These constructs can be combined to form complex query expressions (see Example 2 below). The supported constructs include (refer to Table 1):

Singleton (S): The simplest construct, representing fixed numeric value(s) at specific positions within the query object. Traditional techniques that operate on exact sequences typically support (only) this construct.

Range (R): Specifies a valid range interval for a value at a given position in the query object. As shown in Table 1, range boundaries may be inclusive or exclusive.

Pattern (P): A semantically rich construct that captures trends across sequences of values (e.g., increasing, decreasing). This construct is defined using four parameters (see Table 1). The **domain** parameter specifies the allowable range of values in the pattern, the **direction** parameter indicates the trend as "+" (increasing), "-" (decreasing), "=" (steady), and "?" (unconstrained), and the **length** parameter specifies the desired sequence length, either as a fixed value or a range. The fourth parameter, referred to as **magnitude**, is an optional parameter that specifies the step size (value difference) between consecutive values in the pattern.

Wildcard (W): Allows for any value to be at specific positions in the query object. Under whole-matching semantics, where the query object and the stored time series must be of equal length, any unspecified positions in the query are implicitly treated as wildcards. Under subsequence-matching, positions outside the matched window are unconstrained by definition, so no trailing wildcard is needed (see the two forms in Example 2).

Example 2 (ECG QRS-and-T-wave morphology): *Consider an electrocardiogram (ECG) lead-II recording sampled at* 500 *Hz, whose normal heartbeat is a P-wave baseline near* 0 *mV, a QRS spike, and a T-wave [24]. A clinician may issue (i) a* ***full-beat template match*** *flagging beats whose QRS-and-T morphology deviates from a healthy template, or (ii) a lighter-weight* ***QRS-spike scan*** *locating only the ventricular spike (R-upstroke and S-downstroke) anywhere in the recording. These map onto a whole-matching form and a subsequence-matching form, respectively.*

Whole-matching form (the query exactly fills the 128-sample stored segment; positions are locked, with a trailing wildcard absorbing the post-T baseline):

$$\langle \underbrace{|[-0.05, 0.05] \overset{=}{} (15)|}_{(a)}, \underbrace{|[-0.30, 0.00] \overset{-}{} (5)|}_{(b)}, \underbrace{|[-0.30, 1.50] \overset{+}{_{[0.05,0.20]}} (12)|}_{(c)}, \underbrace{|[-0.50, 1.50] \overset{-}{_{[-0.20,-0.05]}} (12)|}_{(d)}, \underbrace{|[0.00, 0.40] \overset{?}{} (8, 15)|}_{(e)}, \underbrace{W}_{(f)} \rangle$$

Subsequence-matching form (just the QRS spike — the R-upstroke and S-downstroke constructs from above, reused in isolation; the pattern matches any contiguous window of the host series, with positions outside that window unconstrained):

$$\text{SUBSEQ} \langle \underbrace{|[-0.30, 1.50] \overset{+}{_{[0.05,0.20]}} (12)|}_{(c)}, \underbrace{|[-0.50, 1.50] \overset{-}{_{[-0.20,-0.05]}} (12)|}_{(d)} \rangle$$

The constructs are: (a) A **"P"** construct with a steady (=) trend of length 15 and mV in $[-0.05, 0.05]$: the pre-QRS isoelectric baseline (tail of the PR-segment) — *whole-matching form only*, (b) A **"P"** construct with a decreasing trend of length 5 and mV in $[-0.30, 0.0]$: the small Q-wave dip — *whole-matching form only*, (c) A **"P"** construct with an increasing trend of length 12, mV in $[-0.30, 1.50]$, and an inter-sample magnitude in $[0.05, 0.20]$ mV: the R-wave upstroke, the sharp ventricular spike — *used in both forms; first half of the QRS spike*, (d) A **"P"** construct with a decreasing trend of

Table 1: Description of TSseek Query Constructs

Type	Desc.	Parameters	Parameter Desc.
Singleton (S)	Expresses an explicit number value	v	Self explanatory
Range (R)	Expresses an allowed range for a value with inclusive "[ ]" or exclusive "( )" boundaries	Range of values, e.g., $[v_1, v_2)$	Self explanatory
Pattern (P)	Expresses a trend over a sequence of values, e.g., increasing, decreasing, or steady sequence of values.	(domain) $\frac{\text{direction}}{\text{magnitude}}$ (length)	**domain**: Allowed values within the pattern. Takes the form of a Range construct. **direction**: + (increasing) − (decreasing) = (steady) ? (unspecified) **length**: The length of the pattern. Takes the form of either a Singleton (for fixed-length patterns), or Range (for varying-length patterns). **magnitude**: The step between consecutive values in the pattern. Can be expressed in absolute or relative terms. Takes the form of either a Singleton (for fixed-length step), or Range (for varying-length step).
Wildcard (W)	Expresses *any value is allowed*	$*$ or $(*, \text{length})$	**length**: The length of the wildcard values. It can be either a Singleton (for fixed-length wildcard), or Range (for varying-length wildcard).

length 12, mV in $[-0.50, 1.50]$, and an inter-sample magnitude in $[-0.20, -0.05]$ mV: the S-wave downstroke that drops past baseline — *used in both forms; together with (c) forms the QRS spike that the subsequence form scans for*, (e) A **"P"** construct with an increasing trend of length between 8 and 15 and mV in $[0.0, 0.40]$: the T-wave upstroke, whose duration varies with heart rate — *whole-matching form only*. Finally, (f) appears *only in the whole-matching form*: a **"W"** construct extends through the remainder of the 128-sample window (covering the post-T-wave baseline). Under subsequence matching, the host series is unconstrained outside the matched QRS-spike window — the surrounding P-wave, T-wave, and baselines may take any shape.

TSseek Query Language: For ease of use and thus likelihood of adoption, queries in TSseek can be expressed in a SQL-like declarative language. The following example illustrates such syntax.

Example 3: *The two query forms in Example 2 translate into the SQL-like declarative language as follows. The whole-matching form uses the* MATCHES *operator with all six constructs (including the trailing wildcard); the subsequence-matching form uses* MATCHES_SUBSEQUENCE *with only the QRS-spike constructs (c) and (d).*

```
-- Whole-matching form (Example 2, top)
SELECT * FROM <ECG Dataset>
WHERE MATCHES (
  PATTERN(DOM=[-0.05,0.05], DIR='=', LEN=15),
  PATTERN(DOM=[-0.30,0.00], DIR='-', LEN=5),
  PATTERN(DOM=[-0.30,1.50], DIR='+', MAG=[0.05,0.20], LEN=12),
  PATTERN(DOM=[-0.50,1.50], DIR='-', MAG=[-0.20,-0.05], LEN=12),
  PATTERN(DOM=[0.00,0.40], DIR='+', LEN=[8,15]), WC([*]))

-- Subsequence-matching form (Example 2, bottom)
SELECT * FROM <ECG Dataset>
WHERE MATCHES_SUBSEQUENCE (
  PATTERN(DOM=[-0.30,1.50], DIR ='+', MAG=[0.05,0.20], LEN=12),
  PATTERN(DOM=[-0.50,1.50], DIR ='-', MAG=[-0.20,-0.05], LEN=12))
```

Notice that the TSseek query object is composed of an ordered sequence of query constructs, together representing a set of conjunctive predicates that must all be fulfilled by a corresponding database time series object.

5 TSSEEK INDEXING FRAMEWORK

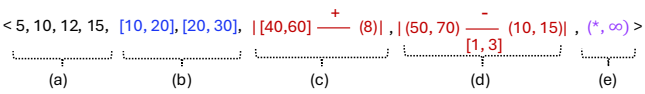
For large-scale datasets comprised of hundreds of millions of time series objects, efficient indexing techniques are essential to circumvent the prohibitive costs associated with naive search strategies such as full dataset scans [11, 49, 54, 60]. Intrinsic to these indexing techniques are the *feature extraction* methods that generate compact feature-based representations of the data in lower-dimensional spaces, upon which index structures are subsequently built [13, 20, 31, 37, 60].

However, conventional feature extraction techniques and their corresponding index structures cannot be directly applied in the context of TSseek. This is because the query object in TSseek is not a standard time series object (see Examples 2 and 3). Consequently, representations such as PAA [31], SAX [37], or Pivot-based methods [44, 62, 74] are not perfectly suitable for such queries. The fundamental challenge, therefore, lies in devising feature representations for both the dataset's time series objects and the query objects that are mutually compatible—ensuring they can be compared within a shared representation space. Only then, an index be constructed over the extracted representation and effectively exploited during query execution.

In Sec. 5.1 and 5.2, we present the TSseek's feature extraction and index structure, respectively, specifically designed to overcome the aforementioned limitation.

5.1 TSseek Feature Extraction

Key Insight. A close examination of the supported query constructs (e.g., singletons, ranges, and patterns) reveals that they primarily constrain the search space to specific regions within a two-dimensional domain, where the x-axis corresponds to the temporal dimension (equivalent to the sequence position), while the y-axis corresponds to the value dimension. For instance, consider the following representative query of length 24, composed of singletons, ranges, and trend patterns:



This query can be naturally visualized within the two-dimensional space, as shown in Fig. 2. Certainly, variable-length patterns, large ranges, and wild card segments will add to the complexity of such two-dimensional constraints, e.g., the bounding rectangles for segments **d** and **e** could have a variable end and/or start, respectively. Such complexities will be accommodated for during query processing (Sec. 6).

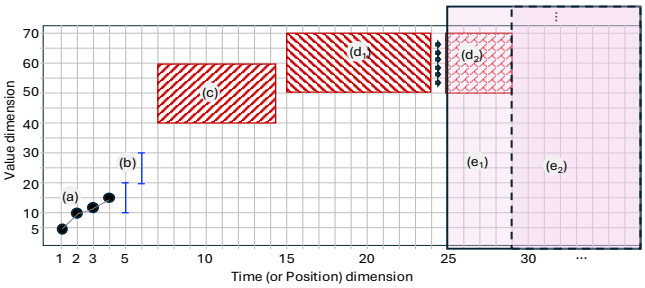


Figure 2: Two-dimensional visualization for an illustrative query object.

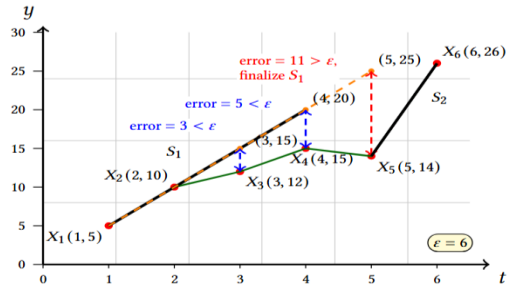


Figure 3: Example of piecewise linear approximation on sample time series.

Building on this observation, our key insight is to extract features from time series objects by approximating them as sequences of two-dimensional line segments. This ensures that both the dataset objects and the query representations are mapped into the same feature space. Consequently, spatial index structures can be efficiently employed to accelerate query processing.

TSseek Feature Extraction Algorithm. The segmentation algorithm transforms each time series object into an ordered sequence of segments, as outlined in Alg. 1. For a time series t, the algorithm employs Piecewise Linear Approximation (PLA) [33, 35] left-to-right [15]. The first two points form the initial segment (Lines 3–5); it then iteratively extends the segment to the next point while the approximation error stays within ε (Lines 8–12). Otherwise, the segment is terminated and a new one initiated (Lines 13–18).

Example 4: *As illustrated in Fig. 3, let* $\varepsilon = 6.0$ *and consider the first 6 points of a time series:* $\langle(1,5),(2,10),(3,12),(4,15),(5,14),(6,26)\rangle$. *The initialization step creates a segment connecting Points* X_1 *and* X_2. *The segment will extend to Points* X_3 *and* X_4 *because the computed error is* 3.0 *and* 5.0, *respectively, which is less than* ε. *Then, at Point* X_5, *the computed error is* $11.0 > \varepsilon$, *and hence, the* 1^{st} *line segment ends at* X_4, *and a new line segment starts by connecting* X_5 *and* X_6.

As illustrated in Example 4, Alg. 1 retains the initial slope of a line segment once it is initialized, thereby eliminating the need for re-computation with each newly processed point. This simple design achieves efficiency, since each data point is examined only once, resulting in a linear time complexity of $O(n)$, where n denotes the length of the time series. Further, the threshold ε constrains the maximum permissible approximation error for any point in the sequence. This provides a tunable balance between the number of

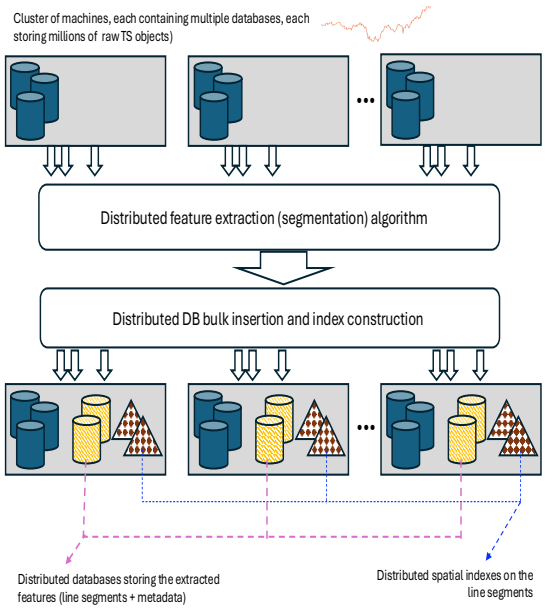


Figure 4: TSseek Storage and Indexing Layer.

segments and the approximation's accuracy. We adopt a data-driven strategy that uses sampling data statistics during preprocessing to autonomously select ε in a manner that is robust across datasets of varying scales and noise characteristics, ensuring an effective trade-off between efficiency and precision.

5.2 TSseek Index Structure

Distributed Raw Dataset. TSseek is implemented as a fully distributed system on top of Apache Spark [76]. The storage layer leverages Postgres DBs instead of HDFS for efficient spatial storage and indexing (see Fig. 4). The time series dataset is partitioned and distributed across the databases over the cluster nodes. Each time series object $X =< x_1, x_2, \ldots, x_m >$ is stored as a single object (single tuple in a database table). A single table contains millions of time series objects. The entire dataset typically spans a large number of such tables.

Distributed Spatial Index (TSseek-X). The indexing framework of TSseek, which we refer to as **TSseek-X**, is built on a distributed spatial index constructed from the line segments of each time series. The **construction process,** summarized in Alg. 2 and illustrated in Fig. 4, is executed in a fully distributed manner. For each time series object X with a unique identifier X_{id}, Alg. 1 is called for extracting X's line segments list L_X (Line 4). The list L_X is then hashed using X_{id} into a target partition $P_{h(X_{id})}$ (Line 5). This hashing strategy ensures that all line segments of the same time series remain co-located, while also maintaining balanced distribution across partitions.

After the line segments are distributed to their assigned partitions, each partition is bulk-inserted into a PostgreSQL table (Line 9). PostgreSQL is selected because of its spatial extension PostGIS [53], which provides efficient storage, indexing, and querying of geometric objects. In our implementation, time series segments are represented using the `LINESTRING` data type, which is well-suited

Algorithm 1: Piecewise Linear Approximation for Time Series Data Segmentation

Input: Time Series Data X, max-allowed-error threshold ε
Output: List of linear segments L_X

begin
  // Initialization
  Construct initial segment S_1 by connecting X_1 and X_2
  Set current segment index $i \leftarrow 1$
  Initialize $j \leftarrow 3$ // The 3rd reading in the time series
  while $j \leq length(X)$ **do**
    Project X_j onto the extended line of S_i to get projection point P_j
    // Error Evaluation
    Compute the approximation error E as the distance from X_j to P_j
    if $E < \varepsilon$ ***OR*** $j = length(X)$ **then**
      Extend S_i to point P_j // maintains same slope
    else
      Finalize current segment S_i
      Construct S_{i+1} by connecting X_j and X_{j+1}
      Increment segment index $i \leftarrow i + 1$
      Increment $j \leftarrow j + 1$ // skip the next reading
    Increment $j \leftarrow j + 1$ // jump to the next reading
  return *and store all segments* S_i *in list* L_X

Algorithm 2: Spatial Index Construction for Time Series Segments

Input: Time series dataset D stored in HDFS, threshold ε
Output: Distributed spatial index on cluster machines

begin
  // Segmentation and Distribution
  foreach *time series* X *in dataset* D **do**
    $L_X \leftarrow$ Algorithm 1 (X, ε)
    Compute target partition $P = \text{hash}(X_{id})$
    Distribute segments L_X to partition P across the cluster
  // Local Index Construction
  foreach *partition* P **do**
    Table $T_P \leftarrow$ Bulk-insert P into PostgreSQL table
    Create GiST R-tree index on the LINESTRING geometry column in T_P
  Store partition metadata for query routing

for modeling line segments. An R-Tree index is then built on this column within PostGIS to enable efficient spatial query processing.

Collecting Data Statistics. During index construction, TSseek collects statistics capturing the distribution of segments within the search space. These statistics serve an important role in query processing, as they are used to estimate query selectivity for each of TSseek's query constructs. To achieve this, the search space—where the x-axis corresponds to time and the y-axis corresponds to values (as illustrated in Fig. 2)—is partitioned into an M*N grid. We then maintain the count of intersecting line segments per grid cell.

Computing statistics over the entire dataset (every segment's grid-cell intersections) does not scale, so we instead use a random sample: for each sampled segment, all intersecting grid cells are identified and their counts incremented. These statistics later guide query distribution and execution (Sec. 6.2).

6 QUERY PROCESSING

Per Def. 4, TSseek aims to find the exact answer set matching the given query object Q. Both whole-matching and subsequence-matching queries follow through the same three-step pipeline: *query rewriting and simplification*, *selectivity estimation*, and *distributed query execution* as presented next.

6.1 Query Rewriting and Simplification

We employ a set of rewriting and simplification rules designed to enhance the efficiency of query execution. Representative examples of these rules are described below. Throughout this subsection we use a representative query of length 24 (visualized in Fig. 2) as a running illustration of the rewriting rules.

Resizing variable-length and wildcard patterns. Variable-length and wildcard patterns can be resized—or, in some cases, entirely eliminated—to enforce the *whole-match* constraint. This constraint requires the length of the query object to match the length of the dataset objects. For instance, consider the illustrative query depicted in Fig. 2. Given a time series length of 24, the query can be reformulated as follows.

$$< 5, 10, 12, 15, \ [10, 20], [20, 30], \ |\,[40,60] \xrightarrow{+} (8)|\ , |\,(50, 70) \xrightarrow[[1, 3]]{-} (10)|\ >$$

Similarly, under the same length of 24, the following query:

$$<|\,[10,15] \xrightarrow{+} (10)|,\ |\,[15,15] \xrightarrow{=} (5)|,\ |\,(20, 40) \xrightarrow[[1, 3]]{-} (5, 10)|,\ |\,[30, 40] \xrightarrow{+} (2,8)|>$$

can be re-written as below. More specifically, the minimum length requirement from all patterns in the query is 10 + 5 + 5 + 2 = 22. Therefore, the variability in the last two patterns can be constrained to the remaining 2 positions. The re-written query is shown in Fig. 5(a).

$$<|\,[10,15] \xrightarrow{+} (10)|,\ |\,[15,15] \xrightarrow{=} (5)|,\ |\,(20, 40) \xrightarrow[[1, 3]]{-} (5, 7)|,\ |\,[30, 40] \xrightarrow{+} (2,4)|>$$

Superset coverage of overlapping regions. The boundaries formed by the ending position of a variable-length pattern and the starting position of the subsequent pattern define a region referred to as the *overlapping region* (see Fig. 2 and Fig. 5(a)). Ultimately, this region is associated with either the preceding or the succeeding pattern.

To simplify the *overlapping region* for potential exploitation later in selectivity estimation and query optimization, we rewrite such regions to the superset ranges covering the overlapping patterns. This ensures the candidates retrieved from the index are a correct superset of the actual answer set. The refinement phase during query execution (Sec. 6.3) will then filter out any false-positive candidates.

As an example, referring to the region (d_2 & e_1) illustrated in Fig. 2, the superset coverage for these positions is e_1. Hence the

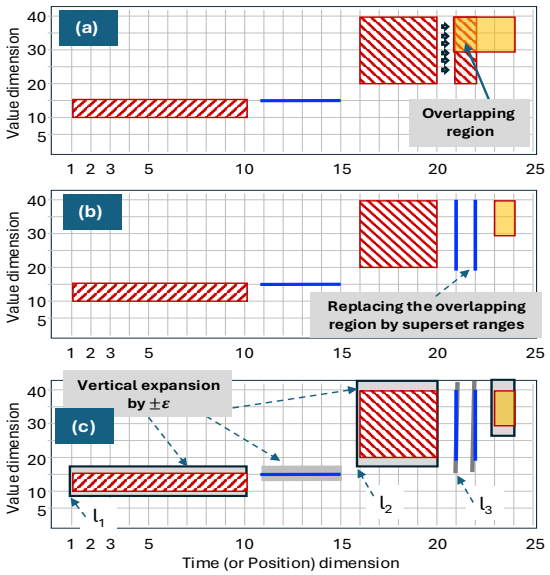


Figure 5: Example of query re-writing and expansion.

query, for the index retrieval purposes, can be simplified as stated below.

< 5, 10, 12, 15, [10, 20], [20, 30], | [40,60] —(+)— (8)| , | (50, 70) —([1, 3])— (10)|, (*, ∞) >

Similarly, the overlapping region in the query depicted in Fig. 5(a) can be simplified as illustrated in Fig. 5(b). The two positions in the overlapping region either belong to the decreasing pattern having range (20,40) or the increasing pattern having range (30,40). Therefore, their superset coverage is the range of (20,40), but with an unconstrained direction. The simplified expression is given below.

< | [10,15] —(+)— (10)|, | [15,15] —(=)— (5)|, | (20, 40) —([1, 3])— (5)|, [20, 40], [20, 40], | [30, 40] —(+)— (2)| >

Early termination due to conflicting constraints. The system employs early termination for queries that are guaranteed to yield no results due to conflicting constraints. This is applied at both the individual pattern level and the overall query level. For instance, at the pattern level, consider the following query pattern: `PATTERN(DOMAIN = (10,20), DIRECTION = '+', MAGNITUDE = 3, LENGTH = 10)`. This pattern is unsatisfiable because it is not possible to fit 10 values, each increasing by a step of 3, within the specified range of (10, 20).

6.2 Selectivity Estimation for Index Lookup

Conversion rules. Each construct of a query object $Q = \langle q_1, q_2, \ldots, q_m \rangle$ is mapped to a spatial probe over the segment index according to the following rules. Let p denote the construct's start position, L its length, $[v_{\min}, v_{\max}]$ its value-domain bounds, and ε the max-error tolerance from Sec. 6.4.

- A singleton $S(v)$ at position p contributes the point-probe rectangle $[p,\ p] \times [v-\varepsilon,\ v+\varepsilon]$.
- A range $R[v_{\min}, v_{\max}]$ at position p contributes $[p,\ p] \times [v_{\min}-\varepsilon,\ v_{\max}+\varepsilon]$.
- A pattern $P^d[v_{\min}, v_{\max}]_L$ with direction $d \in \{+, -, =, ?\}$ at position p contributes the strip-probe rectangle $[p,\ p+L-1] \times [v_{\min}-\varepsilon,\ v_{\max}+\varepsilon]$ together with the metadata filter slope $d\,0$ (applied when $d \in \{+, -, =\}$; omitted when $d=?$); if the pattern also specifies a magnitude range $[m_{\min}, m_{\max}]$, the additional inter-sample filter $m_{\min} \le \Delta y \le m_{\max}$ is applied (signed: positive bounds for direction +, negative bounds for direction −).
- A wildcard W over positions $[a, b]$ imposes no constraint and is omitted from the probe set.

The ε inflation on every value-range bound enforces the completeness guarantee discussed below: any segment whose original (unsmoothed) values fall within the construct's intended range is guaranteed to intersect the inflated rectangle, so no candidate is lost during pruning.

Referring to the visual representation of a query, each time window (including as default a window of length zero, i.e., a time position) along the x-axis can form a spatial query for probing **TSseek-X**. The process of determining which spatial query to execute involves two main steps:

Step 1 (ε expansion): As outlined in the feature extraction and segmentation method (Alg. 1), the maximum permissible approximation error is ε (see Fig. 3). To guarantee that the index lookup retrieves a superset of the correct query results, this approximation error must be accounted for during query formulation. Accordingly, each query component is expanded vertically by $\pm\varepsilon$.

Completeness guarantee. During segmentation, each segment's maximum vertical approximation error is bounded by ε. Therefore, a segment with approximated range $[y'_{\min}, y'_{\max}]$ may represent original points in $[y'_{\min} - \varepsilon, y'_{\max} + \varepsilon]$. By expanding each query window $[y_{\min}, y_{\max}]$ vertically by $\pm\varepsilon$ to $[y_{\min} - \varepsilon, y_{\max} + \varepsilon]$, we guarantee spatial intersection with all segments whose original points fall within the query range. Importantly, errors do not compound across segments because each segment is independently approximated from the raw data–the ε bound applies to each segment, not cumulatively. This ensures completeness (no false negatives) for queries of any length, though false positives may occur and are filtered during subsequent refinement.

Step 2 (Selectivity estimation): The second step involves selecting from the set of potential queries derived from the user's input, the specific spatial query to execute on the TSseek index. The goal is to identify the query with the highest selectivity, thereby enhancing performance and ensuring efficient execution.

For each non-wildcard construct, probing only the left-most and right-most boundary positions (each ε-expanded vertically) suffices: a segment covering the construct's full span intersects *both* boundary windows, so interior positions add no discriminating power. This shrinks the candidate-probe pool from $|Q|$ to roughly $2\,k_{\text{constructs}}$ (e.g., a length-24 query with 5 non-wildcard constructs yields 10 candidate probes instead of 24), from which we select the single most-selective construct. A series qualifies only if hit by every boundary window: per table, the probe id-sets are intersected, then unioned across the N tables, with remaining false positives removed during FSA refinement.

Finally, for each candidate spatial query, we estimate its selectivity using the grid-based data statistics collected during the index construction step. We then choose the most selective one.

6.3 Distributed Query Execution

The end-to-end query execution workflow is summarized in Alg. 3. Given a query Q, the process begins with query rewriting (per the rules described above), followed by selectivity estimation, which selects the *single most-selective construct* c^* as the index probe (Sec. 6.2). The remaining constructs are not probed at the index stage; the full conjunctive query—all constructs of Q—is instead enforced by the FSA refinement step. The next phase distributes c^*'s boundary probe(s) across all machines to perform *spatial intersection* queries over the database tables.

Two key design decisions motivate the (N)-tables/(M)-machines architecture. First, ($N > M$), ensuring that segments are distributed across a greater number of logical partitions than there are physical hosts. This enables spatial-index scans to be parallelized at a finer granularity than would be possible using host-level parallelism alone. Second, during preprocessing, the segments associated with each table are batch-loaded into the PostgreSQL instance running on the host assigned to that table, ensuring co-location of the table and its corresponding segment data on disk. Spark orchestrates distributed preprocessing and cross-partition aggregation, while PostgreSQL with PostGIS executes indexed spatial retrieval locally on each partition.

The output of these queries consists of the identifiers of matching line segments, along with their associated metadata (e.g., slope and the *parent time-series id*, which links each segment back to the original time series from which it was extracted during preprocessing). At this stage, the algorithm incorporates the pattern direction specified in the query (if provided) and compares it with the slope direction of the retrieved line segments to further refine the candidate set. For instance, if the selected probe carries an increasing-trend constraint, segments whose stored slope is not increasing are discarded directly within the index scan.

Slope-based filtering is essentially free, happening entirely within PostgreSQL's index layer: each segment's pre-computed slope and fluctuation flag are indexed columns. PostgreSQL's query optimizer automatically combines the spatial GIST index scan with the B-tree slope index, eliminating a lot of false positives. Once this local filtering is applied, the per-table boundary-probe result sets are intersected (keeping series hit by every boundary window) and then unioned across the N tables into the candidate set carried forward.

In the final stage, the algorithm retrieves the raw time series data corresponding to the candidate IDs and immediately evaluates them against the exact query using a DFA-based refinement test. The time series data are stored in distributed PostgreSQL tables across the cluster nodes. Each table is indexed using a B-tree on the time series IDs to enable efficient lookups. Retrieval queries utilize these B-tree indexes to minimize network round trips, with each index occupying approximately 40 MB and fully residing in memory.

Retrieval and refinement are executed in a streaming pipeline: candidate IDs are fetched in batches, and each batch is refined on the fly using DFA-based pattern matching, eliminating the need for intermediate storage. Only matching results are retained in memory. This process is distributed across Spark executors, where each PostgreSQL table is processed independently and in parallel, with DFA refinement executed locally on the corresponding data storage node.

6.4 Data-Driven Configuration

Threshold ε. The max-error tolerance ε used by the segmentation algorithm (Alg. 1) is calibrated to each dataset. From a small sample of raw values we compute $\text{span}_{90} = p_{95} - p_5$, the difference between the 95th and 5th percentiles of the value distribution, which captures the typical amplitude of the central 90% of the data. We then set $\varepsilon = \alpha \cdot \text{span}_{90}$ for a user-chosen scaling factor $\alpha \in (0, 1]$: a small α preserves fine detail of the data, a large α tolerates more variation. Sec. 7 reports how α trades off setup and query-processing time.

Grid cell size. During the same sample pass, we record for each emitted segment its horizontal extent $|\Delta x|$ (in samples) and vertical span $|\Delta y|$ (value change). We then choose grid-cell width and height so that a typical ($|\Delta x|$, $|\Delta y|$) pair spans about two cells along each axis. This keeps most segments confined to a small neighborhood of cells—which improves index selectivity during query processing—and lets the grid size adapt to the observed data rather than relying on hand-tuned constants.

6.5 Subsequence-Matching: Filter and Refinement Specialization

We now describe the two places where the pipeline of Sec. 6.2 and 6.3 above specializes for subsequence-matching queries. The offline index and segmentation layers are reused unchanged; only the SQL filter emitted to PostgreSQL (Stage 1) and the placement check performed by the refinement DFA (Stage 2) differ. A subsequence query Q consists of $k \geq 1$ sub-patterns, each of which must match a contiguous window of the host series; for multi-pattern queries ($k \geq 2$) the relative positions of those windows are governed by the query's ordering and gap constraints. The two stages are now detailed in turn:

(Stage 1) PostgreSQL composite filter. A subsequence query is decomposed into $k \geq 1$ *sub-patterns* $p_1, \ldots, p_k$, each of which is a TSseek trend-pattern construct. Each sub-pattern is compiled into a spatial-index probe predicate ϕ_i over the segment table. Rather than issue k separate probes and intersect their per-id outputs, the executor fuses them into a single SQL scan with a `GROUP BY ts_id HAVING` clause that ANDs a `bool_or(`ϕ_i`)` aggregate per sub-pattern (`bool_or` returns `TRUE` iff any row in the group satisfies the predicate):

```
SELECT ts_id FROM segments
GROUP BY ts_id    HAVING bool_or(phi_1) AND bool_or(phi_2)
AND bool_or(phi_k);
```

A time-series id is emitted only if every sub-pattern is matched at *some* segment of that series. Crucially: (i) the query does *not* enforce ordering, gap, or relative-position constraints in this stage—those are deferred to refinement; (ii) PostgreSQL evaluates the aggregate in a single GiST + B-tree co-scan over the segment table per partition, so the cost is dominated by one index sweep per node regardless of k; (iii) the metadata-based pruning (e.g., slope

Algorithm 3: TSseek Whole-Matching Query Processing Workflow

Input: Time series dataset D, INDEX(D), Query Q
Output: $F = \{\}$ // The final answer set to Q

begin
- $Q' \leftarrow$ Apply query rewriting rules to Q.
- // Selectivity estimation (Sec. 6.2)
- **foreach** *non-wildcard construct c in* Q' **do**
  - $W_c \leftarrow$ the ε-expanded boundary window(s) of c (one window for a singleton/range; left and right for a fixed pattern).
  - $sel_c \leftarrow$ estimated number of segments intersecting W_c from grid statistics.
- $c^* \leftarrow$ the single most selective construct (smallest sel_c).
- $Cand \leftarrow \emptyset$
- **foreach** *table T among all N tables across the M machines* **do**
  - **foreach** *boundary window w in* W_{c^*} **do**
    - $I_w \leftarrow$ DISTINCT parent time-series ids whose segments intersect w in T (slope/direction-filtered in-DB).
  - $Cand_T \leftarrow \bigcap_{w \in W_{c^*}} I_w$ // per-table intersection: series with segments on every boundary window
  - $Cand \leftarrow Cand \cup Cand_T$ // union across tables
- $TS_{List} \leftarrow$ Retrieve the raw time series objects for $Cand$.
- **foreach** *time series t in* TS_{List} **do**
  - **if** *FSA(Q, t) = true* **then**
    - $F = F \cup t$ // t passes the full-query FSA test.
- Return F

Algorithm 4: TSseek Subsequence-Matching Workflow

Input: Time series dataset D, INDEX(D), subsequence query Q with sub-patterns $p_1, \ldots, p_k$ and a query-class constraint Ω (positioning, length, ordering, gap)
Output: F // the answer set to Q

begin
- $F \leftarrow \emptyset$
- `// Stage 1: PostgreSQL composite filter`
- **for** $i \leftarrow 1$ **to** k **do**
  - $\phi_i \leftarrow$ Compile p_i into a spatial-index predicate (y-range + direction + fluctuation).
- $TS_{Ids} \leftarrow$ Issue distributed composite SQL `SELECT ts_id FROM segments GROUP BY ts_id HAVING bool_or(`ϕ_1`) AND ··· AND bool_or(`ϕ_k`)` over INDEX(D).
- $TS_{List} \leftarrow$ Retrieve raw time-series objects for TS_{Ids} from local DB partitions.
- `// Stage 2: vectorized DFA refinement`
- **foreach** *time series* $t \in TS_{List}$ *of length* n **do**
  - **for** $i \leftarrow 1$ **to** k **do**
    - $PR_i \leftarrow$ prefix sums over t of "value $\in p_i$'s y-range".
    - $PM_i \leftarrow$ prefix sums over t of "transition matches p_i's direction".
  - **if** $DFA_\Omega(t, \{PR_i\}, \{PM_i\}) = TRUE$ **then**
    - $F \leftarrow F \cup \{t\}$
- **return** F

direction) described in Sec. 6.3 is applied to each ϕ_i for free as part of the index scan.

(Stage 2) Vectorized DFA refinement with prefix-sum window check. The candidate id list returned by Stage 1 is shipped to the Spark executor co-located with the data partition. For each candidate series of length n, the refiner precomputes, per sub-pattern p_i, two prefix-sum arrays in $O(n)$ time: counts of positions whose value lies within p_i's y-range, and counts of inter-sample transitions whose sign matches p_i's direction. Given any candidate placement window $[a, a{+}L{-}1]$ for p_i, the test of whether p_i is fully matched on that window—i.e., all L values lie in p_i's y-range *and* all $L{-}1$ transitions follow p_i's direction—reduces to two $O(1)$ subtractions on these prefix sums. The DFA enumerates the legal placements dictated by the query: for a single sub-pattern ($k{=}1$), this is a set of (start, length) pairs constrained by the query's positioning (free, bounded, or fixed start) and length (fixed or dynamic); for multiple sub-patterns ($k{\geq}2$), it is a set of $(s_1, L_1, \ldots, s_k, L_k)$ tuples subject to the query's ordering and gap constraints (e.g., adjacent, fixed gap, range gap, or flexible gap). The $O(1)$ window test is applied at each candidate placement; concrete refinement timings across dataset sizes are reported in Sec. 7.

The algorithm for the subsequence matching mirrors the main flow of Alg. 3 with the integration of Stage-1 filter and the Stage-2 DFA into the execution pipeline. The detailed algorithm is given in Alg. 4.

Design notes. Both stages are tuned for the dominant scaling cost of subsequence search. The composite Stage-1 filter folds k per-sub-pattern probes into a single SQL pass, avoiding the k separate JDBC round trips per partition and the driver-side k-way id-set intersection that a sub-pattern-by-sub-pattern plan would incur. The Stage-2 prefix-sum machinery makes each candidate-window test $O(1)$ regardless of window length, which matters most for queries with many candidate placements (free-position and flex-gap). Together these two design choices account for the speedup over T-ReX [28] reported in Sec. 7.

7 EXPERIMENTAL EVALUATION

7.1 Experimental Methodology and Setup

Cluster Setup & Implementation. All experiments were conducted in our locally deployed cluster consisting of 112 CPU cores of type Intel Xeon E5-2690, each with 16 GB RAM and a total of 3.5 TB SATA hard drive. The cluster runs Debian with Spark 3.5.0 and PostgreSQL-15. All algorithmic modules of TSseek are implemented in a fully distributed fashion on top of Apache Spark. The storage layer uses PostgreSQL DB extended with PostGIS for spatial indexing.

Spark + PostgreSQL + PostGIS. TSseek's runtime combines Apache Spark with distributed PostgreSQL instances (each extended with PostGIS). Spark serves three coordinated roles: (i) distributed feature extraction during preprocessing, where each of the N partitions is fed by a separate Spark task; (ii) *coordination of the distributed PostgreSQL probes*, where Spark issues parallel queries across all partition databases and collects/intersects their candidate id sets; and (iii) *distributed DFA refinement*, where each candidate

batch is verified locally on the partition's data-resident node. PostgreSQL with PostGIS supplies the indexed storage and the GiST spatial index; the single-runtime division of labor avoids reimplementing distributed coordination on top of PostgreSQL or spatial indexing on top of Spark.

Datasets. We evaluate TSseek using three datasets:

(1) ECG (Electrocardiogram) Real-World Dataset. This dataset is derived from a large-scale 12-Lead Electrocardiogram Database for Arrhythmia Study (v1.0) [1], which contains over 40,000 ECG recordings sampled at 500 Hz across 12 leads, with detailed arrhythmia annotations. From each recording, we extract leads I–IV, convert amplitude measurements from μV to mV, and apply a 128-sample (0.256 s) sliding window with 75% overlap (stride = 32) to produce time-series segments.

(2) Random-Walk Benchmark Dataset. This is a widely adopted benchmark for evaluating time series analysis techniques [11, 51, 54, 60, 73, 79]. This dataset contains up to one billion data series objects, each having 128 points.

(3) TSBS Fuel Consumption Synthetic Dataset. The Time Series Benchmark Suite (TSBS) [64] provides synthetic telemetry data. We utilize the fuel consumption metric generated from 4,000 simulated truck devices to assess query scalability under industrial-like workloads. For each dataset, we generate four scale variants containing {25, 50, 100, and 200} million time series objects, each consisting of 128 readings.

Default Parameter Settings. In all our datasets and experiments, the length of the time series objects is set to 128. For the data-driven maximum error threshold ε is set to $\varepsilon_{\text{ECG}} = 0.15$, $\varepsilon_{\text{RW}} = 1.59$, and $\varepsilon_{\text{TSBS}} = 27.18$. In query processing, each reported point in the results is repeated five times, and the average is taken across the five readings. Finally, as the dataset size increases, the number of partitions storing the data should also increase (as in distributed file systems by default). For our database setting, we use 40 tables at the scale of 25 M datasets and double the number of tables as the dataset size doubles.

7.2 Baselines

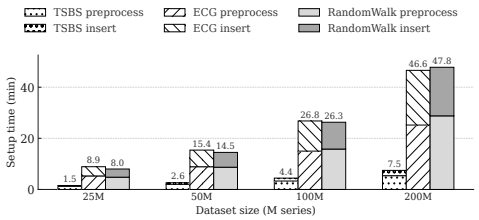
We compare TSseek against four representative techniques:

(1) Deterministic Finite Automata (DFA) [55]: a classical pattern-matching model implemented on Apache Spark; performs a full dataset scan and serves as our exact ground-truth baseline.

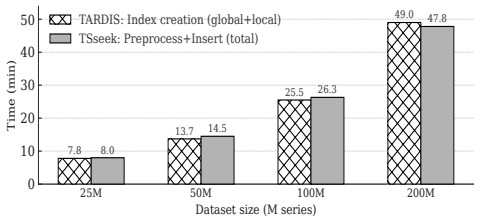
(2) TARDIS [79]: a distributed iSAX-based kNN index on Spark for whole-sequence search. We adapt it to pattern queries by expanding each pattern into its matching instances and re-issuing with a large K, followed by DFA refinement (no false positives, but false negatives are possible).

(3) SEAnet [68]: a learned-embedding approach that trains an autoencoder; embeddings are symbolized to SAX [37] and indexed with an iSAX-style tree [11, 60] for kNN search. Like TARDIS, SEAnet is whole-matching-only—pattern and subsequence queries fall outside its native query model—so we adapt it via the same pattern-expansion-plus-refinement scheme.

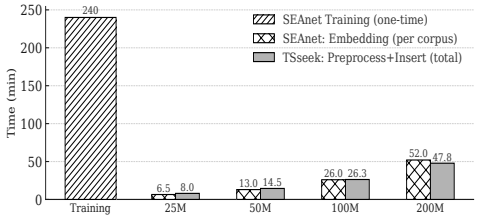
(4) T-ReX [28]: a segment-based subsequence engine that evaluates pattern queries directly over raw series without a persistent index. T-ReX is subsequence-only; we translate each query pattern into its T-ReX equivalent when possible.



(a) TSseek overheads



(b) TSseek vs. TARDIS overheads



(c) TSseek vs. SEAnet overheads

Figure 6: Preprocessing and index (or model) construction cost for different techniques: (a) TSseek; (b) TSseek vs. TARDIS; (c) TSseek vs. SEAnet.

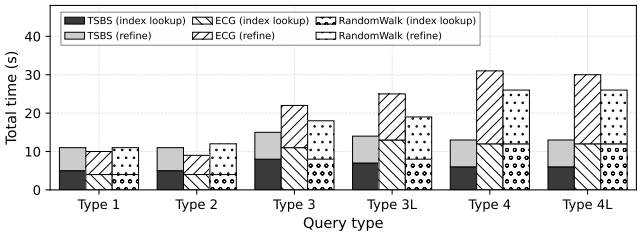
7.3 Evaluation of Preprocessing and Index Construction

Fig. 6a–c present the overheads associated with data preprocessing and index construction across the various data sizes.

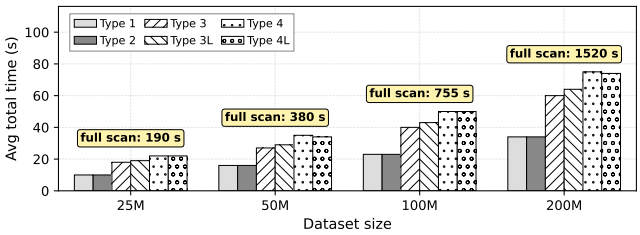
TSseek overheads (Fig. 6a). This cost involves two main phases, the segment creation phase, and insertion and index construction within the PostgreSQL DB. Roughly, the segmentation procedure contributes by 60% to the cost while the insertion and index construction contributes by 40%. Both costs scale linearly as the dataset sizes increase.

TARDIS comparison (Fig. 6b). TARDIS setup cost is comparable to TSseek's and scales similarly with dataset size. However, as will be presented next, since TARDIS is not designed for pattern-based queries, its query accuracy drops to around 30% (vs. TSseek's exact answers) and its query response time is approximately an order of magnitude higher.

SEAnet comparison (Fig. 6c). SEAnet incurs a *one-time* training cost, which can be substantial as illustrated in the figure, plus a per-corpus embedding pass that scales with size. As depicted, the SEAnet's per-corpus embedding cost is comparable to TSseek's

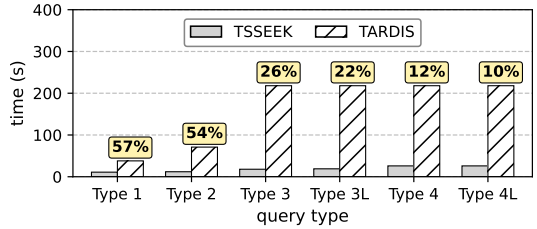


(a) Total query time (index lookup + refine) by type at 25M

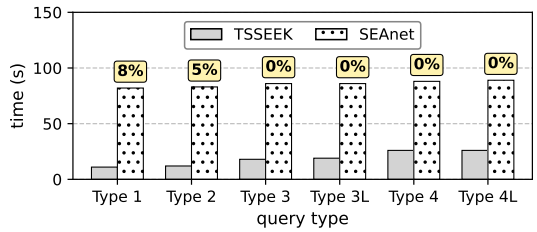


(b) Avg total query time across datasets (Random Walk, ECG, TSBS) vs. size; yellow bubbles show full-scan baseline

Figure 7: TSseek query performance. (a) Total time by query type at 25M (stacked: index lookup + refine). (b) Avg total time across 3 datasets vs. size (25M–200M); yellow bubbles show full-scan baseline.



(a) TSseek vs. TARDIS



(b) TSseek vs. SEAnet

Figure 8: Whole-matching baseline comparison on Random Walk (25M) across six query types. Bars show query time; bubbles above each baseline bar show that baseline's recall (%). (a) TSseek vs. TARDIS. (b) TSseek vs. SEAnet. TSseek recall = 100% on all query types.

total cost. Therefore, for first-time deployment, SEAnet's training cost dominates the overall cost. As will be presented next, SEAnet performance at query time is too low to be practically useful for the problem at hand.

7.4 Evaluation of Query Processing

Query Types (Whole-Matching)

We compose each query from four building blocks: **S** (singleton value), **R** (range for a single reading), $\mathbf{P_F}$ (fixed–length pattern block), and $\mathbf{P_V}$ (variable–length pattern block). In Types 1–4, pattern blocks are relatively *short*; in the "L" variants (3L, 4L) pattern blocks are relatively *long*. Given a target query length L, we allocate length to the four block categories in the stated proportions and then instantiate blocks accordingly.

- **Type 1**: Balanced singletons/ranges/patterns, no variable–length patterns. Length allocation across (S, R, P_F, P_V): $\{\frac{1}{3}, \frac{1}{3}, \frac{1}{3}, 0\}$.
- **Type 2**: All four building block types are included. Length allocation: $\{\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}\}$.
- **Type 3**: No singletons; ranges plus patterns of both fixed and variable length (short). Length allocation: $\{0, \frac{1}{3}, \frac{1}{3}, \frac{1}{3}\}$.
- **Type 3L**: Same composition as Type 3, but with fewer, longer patterns. Length allocation: $\{0, \frac{1}{3}, \frac{1}{3}, \frac{1}{3}\}$.
- **Type 4**: Patterns only (no singletons or ranges), half fixed and half variable length (short). Length allocation: $\{0, 0, \frac{1}{2}, \frac{1}{2}\}$.
- **Type 4L**: Same composition as Type 4 but with fewer (and longer) patterns. Length allocation: $\{0, 0, \frac{1}{2}, \frac{1}{2}\}$.

Per-Query-Type Performance (Whole-Matching). Fig. 7a reports TSseek's per-query-type total time at 25M across the three datasets (TSBS, ECG, RandomWalk). Query Types 1 & 2 include many singletons and ranges, giving strong point-wise selectivity, short candidate lists, and short processing time. In contrast, Types 3 & 4 are pattern-dominated (no singletons in 3/3L; patterns only in 4/4L), loosening early pruning and shifting more work to the refinement phase. The longer variants (3L, 4L) involve longer patterns, further increasing refinement cost.

Scalability vs. Full Scan. Fig. 7(b) reports the average total query time across the three datasets as dataset size grows from 25M to 200M for all six query types. TSseek's total time grows approximately linearly with dataset size and stays under 100 s even at 200M. In contrast, the **full-scan** baseline (yellow bubbles in Fig. 7(b)) is substantially more expensive (190 s at 25M growing to 1520 s at 200M). Overall, TSseek achieves approximately 20× speedup at scale.

Query Evaluation of TSseek vs. TARDIS. As described in Sec. 7.2, we adapt TARDIS via upper- and lower-bound queries on Random Walk (25M); all answers are then verified through a DFA phase, so reported recall counts only false positives. Fig. 8(a) shows the comparison.

TARDIS achieves moderate recall on point-wise Types 1-2 (54-57%) but degrades sharply on pattern-dominated Types 3-4L (10-26%): bounding queries cannot fully cover the search space defined by pattern predicates ($\mathbf{P_F}/\mathbf{P_V}$). Query time follows the same trend: tens of seconds on Types 1-2 but ~218s on Types 3, 3L, 4, and 4L. After z-normalization, Types 1-2 segments (anchored by constants and narrow ranges) yield distinctive SAX words and small candidate sets, while pattern-dominated Types 3-4L segments collapse to similar SAX representations and explode the kNN candidate set. TSseek, natively designed for pattern-based processing with

Table 2: Eleven subsequence-matching query classes used in the evaluation. Column k gives the number of sub-patterns per query.

Class	Group	Description	k
Q1	single	FREE positioning, fixed length	1
Q2	single	FREE positioning, dynamic length	1
Q3	single	BOUNDED start, fixed length	1
Q4	single	BOUNDED start, dynamic length	1
Q5	single	FIXED start, fixed length	1
Q6	single	FIXED start, dynamic length	1
Q7	multi	Adjacent (zero-gap, ordered)	≥ 2
Q8	multi	Fixed gap (ordered)	≥ 2
Q9	multi	Range gap (ordered)	≥ 2
Q10	multi	Flex gap, ordered	≥ 2
Q11	multi	Flex gap, unordered	≥ 2

efficient index retrieval and pruning, attains 100% recall and stable query times (10.7-25.9 s), achieving a **3.5×-12×** speedup (largest gap on pattern-dominated types).

Query Evaluation of TSseek vs. SEAnet. Following the same bounding-query adaptation with K=100,000, we evaluate SEAnet on the same six query types. SEAnet's embeddings are queried via approximate nearest-neighbor search [41]. Training SEAnet on 1M of 25M series required ∼4 h, plus ∼6.5 min to embed the full corpus and ∼2 s per query.

Fig. 8(b) shows that SEAnet achieves only 8% and 5% recall on Types 1 and 2, and 0% on Types 3, 3L, 4, and 4L—learned embeddings cannot represent pattern-based queries. Query times are 82-89 s across all types vs. TSseek's 10.7-25.9 s (**3.4×-7.7×** speedup), with the gap narrowing as pattern content increases. TSseek maintains 100% recall.

We now turn to the subsequence-matching workload, run over the same offline index via the Stage-1/Stage-2 pipeline of Sec. 6.5.

Query Types (Subsequence-Matching). For subsequence matching, the query specifies a pattern that must match some contiguous window inside a stored time series; the pattern need not span the full series length. We organize the workload into eleven query classes (Q1–Q11) that span the design space along three orthogonal axes: *positioning* (free, bounded, or fixed start), *length* (fixed or dynamic), and *composition* (single sub-pattern, or multiple sub-patterns with ordering and gap constraints). Q1–Q6 form the *single-pattern* group; Q7–Q11 form the *multi-pattern* group. Table 2 summarizes the eleven classes.

Each class is instantiated against three datasets (ECG, RandomWalk, TSBS) at four scales (25M, 50M, 100M, 200M time series) and run against both TSseek (DFA, Sec. 6.5) and the T-ReX subsequence engine [28] as baseline.

Query Evaluation of Query Types (Subsequence-Matching). Both systems run on the same cluster and same data partitions, and are verified to produce identical answer sets per query.

Fig. 9 and Fig. 10 show per-class query time at 25M and 200M, respectively. TSseek runs 5–8× faster than T-ReX at 25M; at 200M the gap widens to 9–12× for most classes (Q1–Q9), with multi-flex-gap classes Q10/Q11 reaching 15–19×. At 200M the contrast is visually stark—TSseek's bars shrink to near-negligible against

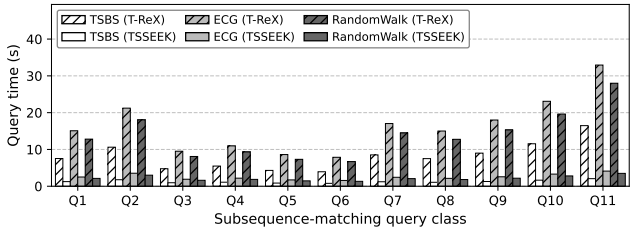


Figure 9: Per-case subsequence query time at 25M (the measured scale). Six bars per class encode dataset by color (white/grey/dark grey = TSBS/ECG/RandomWalk) and method by hatch (diagonal = T-ReX, solid = TSseek).

T-ReX's (note the larger y-axis). The per-class ordering is consistent across datasets: single-pattern fixed-start classes (Q5, Q6) are cheapest (locked start position), bounded variants (Q3, Q4) follow, free-position single-pattern (Q1, Q2) next, and multi-pattern classes (Q7–Q11) are uniformly most expensive. The TSseek-vs-T-ReX gap is largest on multi-pattern queries because T-ReX must enumerate per-pattern matches and verify gap/order constraints by repeated window scans, while TSseek's composite Stage-1 filter collapses all k probes into one and the prefix-sum Stage-2 DFA verifies any window predicate in $O(1)$.

Fig. 11 reports scaling averaged over single-pattern (Q1–Q6) and multi-pattern (Q7–Q11) classes. T-ReX scales linearly (≈ 2× per doubling, non-indexed full-pass) while TSseek scales sub-linearly (≈ 1.5–1.7× per doubling) because the spatial index efficiently prunes most series and the refinement set stays small after the composite filter, widening the gap as data grow.

Fig. 12 makes the speedup widening explicit on ECG: the per-class TSseek-over-T-ReX ratio rises from 5–8× at 25M to 9–12× at 200M for most classes, with the largest gains on the multi-ordered (Q10) and multi-unordered (Q11) flex-gap classes. These two classes are the ones for which T-ReX must explore the most $(p_1, \ldots, p_k)$ window combinations, while TSseek's composite filter prunes at the time-series-id level before any combinatorial work begins. This trend is consistent across all three datasets (speedup reflects query structure, not dataset scale); per-dataset absolute times appear in Fig. 9 and Fig. 11.

Findings. Speedup grows with scale for *every* query class — no class where T-ReX catches up (T-ReX full-pass, TSseek index-driven, per Sec. 6.5). The largest gains on Q10/Q11 align with the multi-pattern flex-gap queries that most motivate subsequence search; the same speedup pattern holds across all three datasets.

7.5 TSseek Ablation Studies

Selecting the Threshold (ε). The threshold ε governs two critical stages of the pipeline of TSseek: setup (preprocessing) and query processing. During preprocessing, it controls how much error we allow when approximating each time series. Thus ε determines both the number of segment records we emit and later bulk-load, and the padding we apply when we construct segment query windows during query processing.

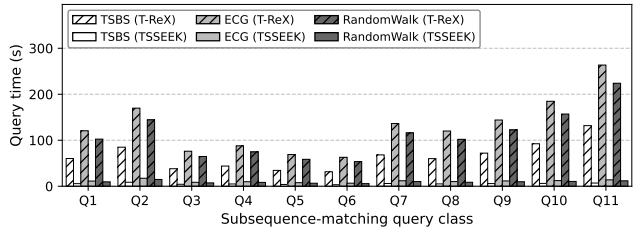


Figure 10: Per-case subsequence query time at 200M. Same encoding as Fig. 9 (color = dataset, hatch = method); the TSseek-over-T-ReX gap widens markedly at scale (note the larger y-axis range).

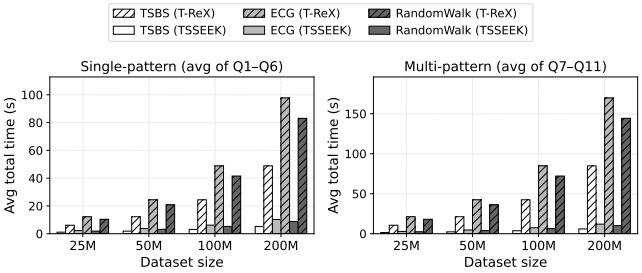


Figure 11: Average subsequence query time vs. dataset size. Left: single-pattern classes (avg of Q1–Q6); Right: multi-pattern classes (avg of Q7–Q11).

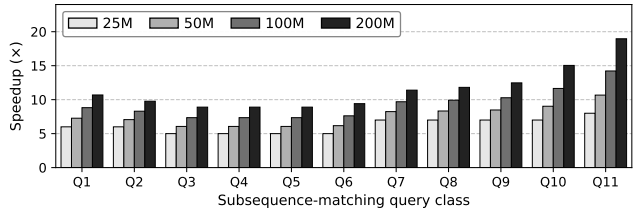


Figure 12: TSseek-over-T-ReX speedup on ECG as a function of query class and dataset size.

To pick a practical tolerance we ran an ablation on TSBS (25 M time series) sweeping $\alpha \in \{0.1, 0.25, 0.5, 0.6, 0.7, 0.75, 1.0\}$. (See Sec. 6.4 for the definitions of α and span_{90}.) Varying α (i.e., the segmentation tolerance $\varepsilon = \alpha \cdot \text{span}_{90}$) trades off setup and query costs: as α increases, setup time drops (fewer segments to emit/insert) but index-lookup time rises (longer segments reduce selectivity), yielding a clear balance point (Fig. 13). Our ablation shows that total cost is minimized when the average segment count is about 7–9 per series. Accordingly, for any new dataset we sample a small slice, sweep a coarse grid of α, and choose the value that yields 7–9 segments per series; this operating point transfers well and balances setup cost with query processing time. In TSBS, this procedure selects $\alpha^* \approx 0.6$.

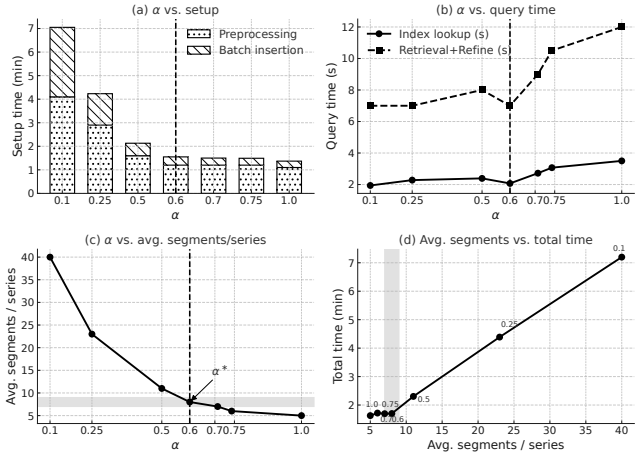


Figure 13: Ablation on α (TSBS), ordered by decision flow: (a) setup time; (b) query time; (c) implied average segments with target band 7–9 and α^*; (d) total cost showing the elbow near ~8 segments.

Slope-Based Pruning. To evaluate the effectiveness of our slope-based filtering mechanism in index lookup, we conducted an ablation study on the TSBS 25M dataset using a query of Type 4L. We compared two configurations: (i) baseline with slope filtering enabled, where PostgreSQL queries include slope constraints (e.g., segment slope > 0 for Increasing patterns), and (ii) ablation with slope filtering disabled, using only spatial intersection. Both experiments used the same infrastructure: 40 distributed PostgreSQL tables across node machines with PostGIS GiST spatial indexes and B-tree indexes on segment slopes.

The results demonstrate that slope filtering provides substantial performance benefits across all query stages. With slope filtering enabled, the system retrieved only 5,926 candidates in 5.6 seconds, completing the retrieval and refinement in 6.0 seconds for a total query time of 11.6 seconds. With slope filtering disabled, the system retrieved 731,705 candidates (about 124× more) in 7.7 seconds, with the retrieval and refinement taking around 17 seconds for a total of 24.7 seconds. The slope-disabled configuration exhibited a significantly slower index lookup (7.7s vs 5.6s) due to increased network transfer overhead and PostgreSQL result set serialization costs for the 124× larger candidate set. This shows that slope filtering not only reduces candidates by 99.2% but also improves PostgreSQL query efficiency, resulting in higher selectivity and much smaller candidate sets for refinement.

Sensitivity to Time-Series Length. We evaluate TSseek's whole-matching performance on ECG-25M at three series lengths: n=64, n=128, and n=256 samples. Fig. 14 reports the per-query breakdown into index-lookup and DFA-refinement times for each length, together with the fitted scaling exponent α above each group (where total time $\propto n^{\alpha}$; $\alpha = 1$ is linear in n, $\alpha < 1$ is sub-linear).

Two scaling regimes drive the per-type behavior. The index-lookup phase grows sub-linearly with n ($\alpha \approx 0.45$ across all query types: at $n = 256$ the lookup cost is only 1.86× its $n = 64$ value, even

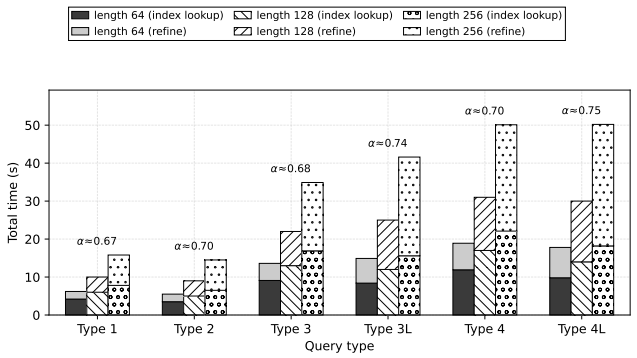


Figure 14: TSseek whole-matching time on ECG-25M across series lengths $n \in \{64, 128, 256\}$. Stacked bars give the index-lookup and DFA-refinement components per query type; the fitted exponent α above each group satisfies total time $\propto n^{\alpha}$ ($\alpha = 1$ is linear in n).

though n itself has grown 4×), because our PLA-based segmentation (Alg. 1) emits a sub-linearly growing number of segments per series. The DFA-refinement phase walks each candidate series end-to-end and therefore scales linearly ($\alpha = 1.0$). The composite exponent annotated above each query group in Fig. 14 sits between these two regimes, pulled toward one or the other by how much pattern content the query carries. Short-pattern queries (Types 1–4), whose constructs are individually narrow or short, generate compact refinement workloads and stay close to the lookup-phase exponent at $\alpha \approx 0.67$–0.70; long-pattern queries (Types 3L and 4L), whose patterns span larger windows, accumulate more refinement work per candidate as n grows and shift toward the refine-phase exponent, reaching $\alpha \approx 0.74$–0.75. Across all six query types total query time stays in the tens of seconds even at $n = 256$, and no query type reaches linear scaling.

8 CONCLUSIONS

We proposed TSseek, a regular-expression-powered search system for distributed time series that lets users specify patterns over trends, value ranges, and wildcard segments. TSseek combines a segmentation-based feature extraction method with **TSseek-X**, a distributed spatial index over the resulting line segments, serving both whole-matching and subsequence-matching queries from one offline index. Experiments across three datasets (ECG, TSBS, Random Walk) at 25M to 200M series demonstrate its effectiveness and scalability. Future work includes variable-rate sampling and combining exact pattern retrieval with approximate k-NN search.

ACKNOWLEDGMENTS

The authors used AI-based language tools (e.g., ChatGPT) solely for grammar and phrasing refinement; all technical content, analyses, experiments, and ideas are the authors' own.

ARTIFACTS

The TSseek implementation—covering the segmentation-based feature extraction, the distributed TSseek-X spatial index, and both the whole-matching and subsequence-matching query processing pipelines—together with the experiment scripts, is publicly available at https://github.com/sslee8778960/TSseek. Owing to their size, the full datasets are not hosted, but all datasets are reproducible from their sources—the ECG recordings from the public PhysioNet database [1], the TSBS telemetry via the TSBS generator [64], and the Random Walk series via the standard random-walk procedure—after conversion to the time-series input format documented in the repository.

REFERENCES

[1] 2020. A Large-scale 12-Lead Electrocardiogram Database for Arrhythmia Study, version 1.0. https://physionet.org/content/a-large-scale-12-lead-electrocardiogram-database-for-arrhythmia-study/1.0.0/.
[2] 2024. POSIX.1-2024 – Portable Operating System Interface (Base Specifications, Issue 8), Regular Expressions. https://pubs.opengroup.org/onlinepubs/9799919799/
[3] Rakesh Agrawal, Christos Faloutsos, and Arun Swami. 1993. Efficient Similarity Search in Sequence Databases. In *Proc. Foundations of Data Organization and Algorithms (FODO) (LNCS, Vol. 730)*. Springer, 69–84.
[4] Rakesh Agrawal, King-Ip Lin, Harpreet S. Sawhney, and Kyuseok Shim. 1995. Fast Similarity Search in the Presence of Noise, Scaling, and Translation in Time-Series Databases. In *Proceedings of the 21st International Conference on Very Large Data Bases (VLDB)*. Morgan Kaufmann, Zurich, Switzerland, 490–501. https://www.vldb.org/conf/1995/P490.PDF
[5] Lars Arge, Mark De Berg, Herman Haverkort, and Ke Yi. 2008. The priority R-tree: A practically efficient and worst-case optimal R-tree. *TALG* 4, 1 (2008), 1–30.
[6] Andreas Bader, Oliver Kopp, and Michael Falkenthal. 2017. *Survey and Comparison of Open Source Time Series Databases.*
[7] Anthony Bagnall and Jason Lines. 2014. An Experimental Evaluation of Nearest Neighbour Time Series Classification. arXiv:1406.4757 [cs.LG] https://arxiv.org/abs/1406.4757
[8] Norbert Beckmann, Hans-Peter Kriegel, Ralf Schneider, and Bernhard Seeger. 1990. The R*-tree: an efficient and robust access method for points and rectangles. In *SIGMOD*, Vol. 19. ACM, 322–331.
[9] Donald J. Berndt and James Clifford. 1994. Using Dynamic Time Warping to Find Patterns in Time Series. In *Proceedings of the 3rd International Conference on Knowledge Discovery and Data Mining (KDD Workshop)*. AAAI Press, Seattle, WA, USA, 359–370.
[10] Siddhartha Bhandari, Neil Bergmann, Raja Jurdak, and Branislav Kusy. 2017. Time Series Data Analysis of Wireless Sensor Network Measurements of Temperature. *Sensors* 17 (05 2017), 1221. doi:10.3390/s17061221
[11] Alessandro Camerra, Themis Palpanas, Jin Shieh, and Eamonn Keogh. 2010. iSAX 2.0: Indexing and Mining One Billion Time Series. In *2010 IEEE International Conference on Data Mining (ICDM)*. 58–67. doi:10.1109/ICDM.2010.124
[12] A. Chakraborti, M. Patriarca, and M. S. Santhanam. [n. d.]. *Financial Time-series Analysis: a Brief Overview.* Springer Milan, 51–67. doi:10.1007/978-88-470-0665-2_4
[13] Kin-Pong Chan and Ada Wai-Chee Fu. 1999. Efficient time series matching by wavelets. In *ICDE*. IEEE, 126–133.
[14] Georgios Chatzigeorgakidis, Dimitrios Skoutas, Kostas Patroumpas, Themis Palpanas, Spiros Athanasiou, and Spiros Skiadopoulos. 2021. Twin Subsequence Search in Time Series. arXiv:2104.06874 [cs.DS] https://arxiv.org/abs/2104.06874
[15] Chia-Shang James Chu. 1995. Time Series Segmentation: A Sliding Window Approach. *Information Sciences* 85, 1–3 (1995), 147–173. doi:10.1016/0020-0255(95)00021-G
[16] Charles L. A. Clarke. 1995. *On the Use of Regular Expressions for Searching Text.* Technical Report. University of Waterloo. https://cs.uwaterloo.ca/research/tr/1995/07/regexp.pdf
[17] Andrew A. Cook, Göksel Mısırlı, and Zhong Fan. 2020. Anomaly Detection for IoT Time-Series Data: A Survey. *IEEE Internet of Things Journal* 7, 7 (2020), 6481–6494. doi:10.1109/JIOT.2019.2958185
[18] Gianpaolo Cugola and Alessandro Margara. 2012. The Complex Event Processing Paradigm. In *Data Management in Pervasive Systems*, F. Colace, M. De Santo, V. Moscato, A. Picariello, F. A. Schreiber, and L. Tanca (Eds.). Springer, 113–133. doi:10.1007/978-3-319-20062-0_6

[19] Hui Ding, Goce Trajcevski, Peter Scheuermann, Xiaoyue Wang, and Eamonn Keogh. 2008. Querying and Mining of Time Series Data: Experimental Comparison of Representations and Distance Measures. *Proceedings of the VLDB Endowment* 1, 2 (2008), 1542–1552. doi:10.14778/1454159.1454226

[20] Christos Faloutsos, M. Ranganathan, and Yannis Manolopoulos. 1994. Fast subsequence matching in time-series databases. In *Proceedings of the 1994 ACM SIGMOD International Conference on Management of Data* (Minneapolis, Minnesota, USA) *(SIGMOD '94)*. Association for Computing Machinery, New York, NY, USA, 419–429. doi:10.1145/191839.191925

[21] Navid Mohammadi Foumani, Chang Wei Tan, Geoffrey I Webb, Hamid Rezatofighi, and Mahsa Salehi. 2024. Series2Vec: similarity-based self-supervised representation learning for time series classification. *Data Mining and Knowledge Discovery* 38, 4 (2024), 2520–2544. doi:10.1007/s10618-024-01043-w

[22] Jeffrey E. F. Friedl. 2006. *Mastering Regular Expressions* (3rd ed.). O'Reilly.

[23] Tak-chung Fu, Fu-lai Chung, Robert Luk, and Chak-man Ng. 2007. Stock time series pattern matching: Template-based vs. rule-based approaches. *Eng. Appl. Artif. Intell.* 20, 3 (April 2007), 347–364. doi:10.1016/j.engappai.2006.07.003

[24] A. L. Goldberger, L. A. N. Amaral, L. Glass, J. M. Hausdorff, P. C. Ivanov, R. G. Mark, J. E. Mietus, G. B. Moody, C. K. Peng, and H. E. Stanley. 2000. PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals. *Circulation* 101, 23 (2000), e215–e220. doi:10.1161/01.cir.101.23.e215 PMID: 10851218.

[25] Antonin Guttman. 1984. R-trees: A dynamic index structure for spatial searching. In *Proceedings of the 1984 ACM SIGMOD international conference on Management of data*. 47–57.

[26] Sylvain Hallé. 2017. From Complex Event Processing to Simple Event Processing. arXiv:1702.08051 [cs.DB] https://arxiv.org/abs/1702.08051

[27] Chaochen Hu, Zihan Sun, Chao Li, Yong Zhang, and Chunxiao Xing. 2023. Survey of Time Series Data Generation in IoT. *Sensors* 23 (2023). doi:10.3390/s23156976

[28] Silu Huang, Erkang Zhu, Surajit Chaudhuri, and Leonhard Spiegelberg. 2023. T-Rex: Optimizing Pattern Search on Time Series. *Proc. ACM Manag. Data* 1, 2, Article 130 (June 2023), 26 pages. doi:10.1145/3589275

[29] Hassan Ismail Fawaz, Germain Forestier, Jonathan Weber, Lhassane Idoumghar, and Pierre-Alain Muller. 2019. Deep Learning for Time Series Classification: A Review. *Data Mining and Knowledge Discovery* 33, 4 (2019), 917–963. doi:10.1007/s10618-019-00619-1

[30] Ibrahim Kamel and Christos Faloutsos. 1993. *Hilbert R-tree: An improved R-tree using fractals.* Technical Report.

[31] Eamonn Keogh, Kaushik Chakrabarti, Michael Pazzani, and Sharad Mehrotra. 2001. Dimensionality reduction for fast similarity search in large time series databases. *KAIS* 3 (2001), 263–286.

[32] Eamonn Keogh, Kaushik Chakrabarti, Michael Pazzani, and Sharad Mehrotra. 2001. Locally Adaptive Dimensionality Reduction for Indexing Large Time Series Databases. In *Proc. ACM SIGMOD Int. Conf. on Management of Data*.

[33] Eamonn Keogh, Selina Chu, David Hart, and Michael Pazzani. 2004. Segmenting Time Series: A Survey and Novel Approach. In *Data Mining in Time Series Databases*. World Scientific, 1–22. doi:10.1142/9789812565402_0001

[34] Eamonn Keogh and Chotirat Ann Ratanamahatana. 2005. Exact indexing of dynamic time warping. *Knowledge and Information Systems* 7, 3 (2005), 358–386. doi:10.1007/s10115-004-0154-9

[35] Eamonn J. Keogh, Selina Chu, David M. Hart, and Michael J. Pazzani. 2001. An Online Algorithm for Segmenting Time Series. In *Proc. IEEE Int. Conf. on Data Mining (ICDM)*. 289–296. doi:10.1109/ICDM.2001.989531

[36] Jessica Lin, Eamonn Keogh, and Stefano Lonardi. 2005. Visualizing and Discovering Non-Trivial Patterns in Large Time Series Databases. *Information Visualization* 4, 2 (2005), 61–82.

[37] Jessica Lin, Eamonn Keogh, Li Wei, and Stefano Lonardi. 2007. Experiencing SAX: a novel symbolic representation of time series. *Data Mining and knowledge discovery* 15 (2007), 107–144.

[38] Michele Linardi and Themis Palpanas. 2020. Scalable Data Series Subsequence Matching with ULISSE. arXiv:2009.10373 [cs.DB] https://arxiv.org/abs/2009.10373

[39] Shuhan Liu, Yuan Tian, Zikun Deng, Weiwei Cui, Haidong Zhang, Di Weng, and Yingcai Wu. 2025. Relation-Driven Query of Multiple Time Series. *IEEE Transactions on Visualization and Computer Graphics* 31, 8 (2025), 4210–4225. doi:10.1109/TVCG.2024.3397554

[40] David C. Luckham. 2001. *The Power of Events: An Introduction to Complex Event Processing in Distributed Enterprise Systems.* Addison-Wesley Longman Publishing Co., Inc., USA.

[41] Yu. A. Malkov and D. A. Yashunin. 2018. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. arXiv:1603.09320 [cs.DS] https://arxiv.org/abs/1603.09320

[42] Konstantinos Mamouras et al. 2024. Efficient Matching of Regular Expressions with Lookaround. In *Proc. ACM SIGPLAN Conference (PLDI/OOPSLA venue family)*. doi:10.1145/3632934

[43] Gautier Marti, Sébastien Andler, Frank Nielsen, and Philippe Donnat. 2016. Clustering Financial Time Series: How Long is Enough? arXiv:1603.04017 [stat.ML] https://arxiv.org/abs/1603.04017

[44] M. L. Micó, J. Oncina, and E. Vidal. 1994. A new version of the nearest-neighbour approximating and eliminating search algorithm (AESA) with linear preprocessing time and memory requirements. *Pattern Recognition Letters* 15, 1 (1994), 9–17. doi:10.1016/0167-8655(94)90095-7

[45] Bhaskar Mitra and Nick Craswell. 2015. Microsoft web search: challenges and opportunities. In *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 3–12.

[46] Young Soo Moon, H. V. Jagadish, Christos Faloutsos, and Joel H. Saltz. 2001. Duality-Based Subsequence Matching in Time-Series Databases. In *Proceedings of the 17th International Conference on Data Engineering (ICDE)*. 263–272.

[47] Cristina Morariu and Theodor Borangiu. 2018. Time series forecasting for dynamic scheduling of manufacturing processes. In *2018 IEEE International Conference on Automation, Quality and Testing, Robotics (AQTR)*. 1–6. doi:10.1109/AQTR.2018.8402748

[48] Tatsuya Nogami et al. 2023. On the Expressive Power of Regular Expressions with Backreferences. *arXiv:2307.08531* (2023). https://arxiv.org/abs/2307.08531

[49] Themis Palpanas. 2015. Data Series Management: The Road to Big Sequence Analytics. *ACM SIGMOD Record* 44, 2 (2015), 47–52.

[50] John Paparrizos and Luis Gravano. 2015. k-Shape: Efficient and Accurate Clustering of Time Series. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data* (Melbourne, Victoria, Australia) *(SIGMOD '15)*. Association for Computing Machinery, New York, NY, USA, 1855–1870. doi:10.1145/2723372.2737793

[51] Botao Peng. 2020. Data Series Indexing Gone Parallel. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. 2059–2063. doi:10.1109/ICDE48307.2020.00244

[52] Perl Community. 2024. *perlre — Perl Regular Expressions.* https://perldoc.perl.org/perlre Perl documentation.

[53] PostGIS Development Team. 2024. PostGIS Documentation. https://postgis.net/documentation Accessed: 2024-04-17.

[54] Thanawin Rakthanmanon, Bilson Campana, Abdullah Mueen, Gustavo Batista, Brandon Westover, Qiang Zhu, Jesin Zakaria, and Eamonn Keogh. 2012. Searching and Mining Trillions of Time Series Subsequences under Dynamic Time Warping. In *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 262–270. doi:10.1145/2339530.2339576

[55] Bala Ravikumar. 1998. Parallel algorithms for finite automata problems. In *Advances in Randomized Parallel Computing*. Springer, 209–239.

[56] Asif Salekin, Md Mustafizur Rahman, and Mohammad Islam. 2012. Composite pattern matching in time series. *Proceeding of the 15th International Conference on Computer and Information Technology, ICCIT 2012*, 173–178. doi:10.1109/ICCITechn.2012.6509784

[57] Nicholas I. Sapankevych and Ravi Sankar. 2009. Time series prediction using support vector machines: a survey. *Comp. Intell. Mag.* 4, 2 (May 2009), 24–38. doi:10.1109/MCI.2009.932254

[58] Jeffrey D. Scargle, Jay P. Norris, Brad Jackson, and James Chiang. 2013. STUDIES IN ASTRONOMICAL TIME SERIES ANALYSIS. VI. BAYESIAN BLOCK REPRESENTATIONS. *The Astrophysical Journal* 764, 2 (Feb. 2013), 167. doi:10.1088/0004-637x/764/2/167

[59] Eli Sherman, Hitinder Gurm, Ulysses Balis, Scott Owens, and Jenna Wiens. 2018. Leveraging Clinical Time-Series Data for Prediction: A Cautionary Tale. arXiv:1811.12520 [cs.LG] https://arxiv.org/abs/1811.12520

[60] Jin Shieh and Eamonn Keogh. 2008. iSAX: indexing and mining terabyte sized time series. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Las Vegas, Nevada, USA) *(KDD '08)*. Association for Computing Machinery, New York, NY, USA, 623–631. doi:10.1145/1401890.1401966

[61] R.H. Shumway and D.S. Stoffer. 2010. *Time Series Analysis and Its Applications: With R Examples.* Springer New York. https://books.google.com/books?id=dbS5IQ8P5gYC

[62] Tomáš Skopal, Jaroslav Pokorný, and Václav Snášel. 2004. PM-tree: Pivoting Metric Tree for Similarity Search in Multimedia Databases. In *Proceedings of ADBIS (Local Proceedings)*. 803–815.

[63] Ken Thompson. 1968. Regular Expression Search Algorithm. *Commun. ACM* 11, 6 (1968), 419–422.

[64] Timescale. [n. d.]. Time Series Benchmark Suite (TSBS). GitHub repository. [Online]. Available: https://github.com/timescale/tsbs. Accessed: Oct. 10, 2025.

[65] Ryota TOMODA and Hisashi Koga. 2025. Section Min-Hash Approximating Time Series Search based on Dynamic Time Warping. *IEICE Transactions on Information and Systems* (01 2025). doi:10.1587/transinf.2024DAP0004

[66] Chen Wang, Xiangdong Huang, Jialin Qiao, Tian Jiang, Lei Rui, Jinrui Zhang, Rong Kang, Julian Feinauer, Kevin A. McGrail, Peng Wang, Diaohan Luo, Jun Yuan, Jianmin Wang, and Jiaguang Sun. 2020. Apache IoTDB: Time-series Database for Internet of Things. *Proceedings of the VLDB Endowment* 13, 12 (2020), 2901–2904. doi:10.14778/3415478.3415504

[67] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *Proc. VLDB Endow.* 14, 11 (July 2021), 1964–1978. doi:10.14778/3476249.3476255

[68] Qitong Wang and Themis Palpanas. 2021. Deep Learning Embeddings for Data Series Similarity Search. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. Association for Computing Machinery, New York, NY, USA, 1708–1716. doi:10.1145/3447548.3467317

[69] Will Wang, Ina Chen, Leeor Hershkovich, Jiamu Yang, Ayush Shetty, Geetika Singh, Yihang Jiang, Aditya Kotla, Jason Shang, Rushil Yerrabelli, Ali Roghanizad, Md Shandhi, and Jessilyn Dunn. 2022. A Systematic Review of Time Series Classification Techniques Used in Biomedical Applications. *Sensors* 22 (10 2022), 8016. doi:10.3390/s22208016

[70] Eugene Wu, Philippe Cudre-Mauroux, and Samuel Madden. 2009. Demonstration of the TrajStore System. *PVLDB* 2 (08 2009), 1554–1557. doi:10.14778/1687553.1687589

[71] Huanmei Wu, Betty Salzberg, Gregory C Sharp, Steve B Jiang, Hiroki Shirato, and David Kaeli. 2005. Subsequence matching on structured time series data. In *Proceedings of the 2005 ACM SIGMOD International Conference on Management of Data* (Baltimore, Maryland) *(SIGMOD '05)*. Association for Computing Machinery, New York, NY, USA, 682–693. doi:10.1145/1066157.1066235

[72] Jiaye Wu, Peng Wang, Ningting Pan, Chen Wang, Wei Wang, and Jianmin Wang. 2019. KV-Match: A Subsequence Matching Approach Supporting Normalization and Time Warping. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. 866–877. doi:10.1109/ICDE.2019.00082

[73] Djamel Edine Yagoubi, Reza Akbarinia, Florent Masseglia, and Themis Palpanas. 2017. DPiSAX: Massively Distributed Partitioned iSAX. In *2017 IEEE International Conference on Data Mining (ICDM)*. 1135–1140. doi:10.1109/ICDM.2017.151

[74] Peter N. Yianilos. 1993. Data Structures and Algorithms for Nearest Neighbor Search in General Metric Spaces. In *Proceedings of the 4th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 311–321. doi:10.1145/313559.313789

[75] Yuncong Yu, Tim Becker, Le Trinh, and Michael Behrisch. 2022. Saxregex: Multivariate Time Series Pattern Search with Symbolic Representation, Regular Expression, and Query Expansion. *SSRN Electronic Journal* (01 2022). doi:10.2139/ssrn.4248588

[76] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J. Franklin, Ali Ghodsi, Joseph Gonzalez, Scott Shenker, and Ion Stoica. 2016. Apache Spark: A Unified Engine for Big Data Processing. *Commun. ACM* 59, 11 (2016), 56–65. doi:10.1145/2934664

[77] Zahra Zamanzadeh Darban, Geoffrey I. Webb, Shirui Pan, Charu Aggarwal, and Mahsa Salehi. 2024. Deep Learning for Time Series Anomaly Detection: A Survey. *Comput. Surveys* 57, 1 (Oct. 2024), 1–42. doi:10.1145/3691338

[78] Jia-wei ZHANG, Hu-sheng LIAO, Hong-yu GAO, and Chang SU. 2018. Research on Complex Event Detection over Streams Supporting Interval Temporal Logic and Regular Expression Pattern Matching. *DEStech Transactions on Engineering and Technology Research* (06 2018). doi:10.12783/dtetr/icmeit2018/23460

[79] Liang Zhang, Noura Alghamdi, Mohamed Y. Eltabakh, and Elke A. Rundensteiner. 2019. TARDIS: Distributed Indexing Framework for Big Time Series Data. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. 1202–1213. doi:10.1109/ICDE.2019.00110

[80] Liang Zhang, Noura Alghamdi, Huayi Zhang, Mohamed Y. Eltabakh, and Elke A. Rundensteiner. 2022. PARROT: pattern-based correlation exploitation in big partitioned data series. *The VLDB Journal* 32, 3 (Oct. 2022), 665–688.

[81] Liang Zhang, Mohamed Y. Eltabakh, Elke A. Rundensteiner, and Khalid Alnuaim. 2024. CLIMBER: Pivot-Based Approximate Similarity Search Over Big Data Series. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. 3933–3946.

[82] Sheng Zhong and Abdullah Mueen. 2024. MASS: distance profile of a query over a time series. *Data Min. Knowl. Discov.* 38, 3 (Feb. 2024), 1466–1492. doi:10.1007/s10618-024-01005-2